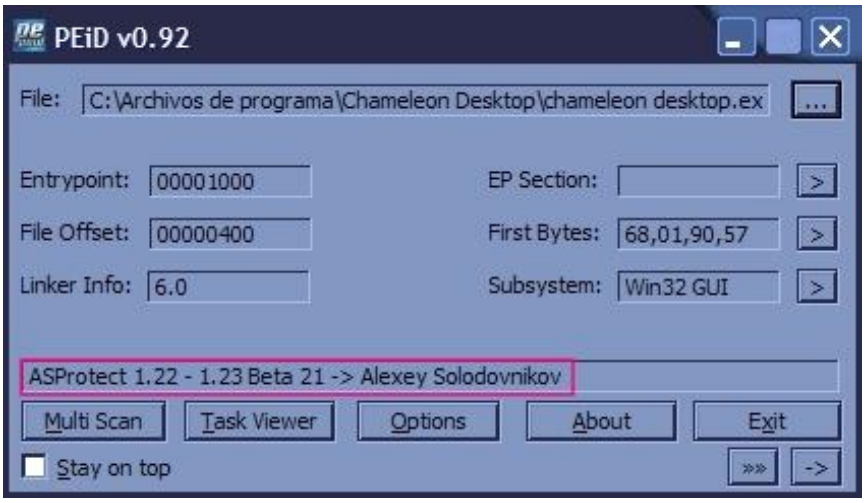
Programa:	Chameleon Desktop 1.3	
Protección:	ASProtect 1.22 + Trial + Inserta LOGO en imagen	
Objetivos:	Desempaquetar, fulminar el Trial y evitar el maldito LOGO ;-)	
URL descarga:	www.softshape.com/chamdesktop	
Dificultad:	Fácil	
Herramientas:	OllyDbg + PEiD + Import Reconstructor.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 09-08-2005

Hola chicos ¿alguna chica? jeje..

Qué tal estamos, volvemos otra vez con otro tuto, en este caso abordaremos ***Chameleon Desktop 1.3***, un sencillo pero elegante programa de retoque fotográfico o algo así, le das la foto y te hace verdaderas virguerias con ella, no creo haber visto ninguno que haga cosas de estas, sí lo típico inserta no se que, añade lo otro y cosas asi pero este no es tan asi, no se es otro estilo, a mi personalmente me gusta bastante. Pero hemos venido aqui a darle publicidad al Chameleon? nos hemos vuelto locos? NOOOOOOO... ;-)

Al Ataque..!!

Abrimos nuestro ya inseparable ***PEiD*** y miramos a ver que sorpresas nos depara el destino en esta mañana de verano, pudiendo estar en la playa ¿? la verdad es que el crackeo te atrapa, jeje.. veamos:



En este caso sobran las presentaciones verdad? un ***ASProtect*** más que conocido por todos, una versión amtigua para los tiempos que corren donde ***ASProtect2*** se ha hecho fuerte en su anti_dump ¿***+NCR*** lo has petao ya? jeje.. saludos compañero, es que está liado con él, lleva tres partes y le queda una cuarta para terminar el tuto realmente muy bueno, sigamos, conociendo este compresor sabemos de antemano que raramente tiene antidump esta versión (ojo no siempre, por ejemplo ***CloneCD 3.2.3.1*** esta empaquetado con esta version y tiene ***antidump***), no hay que fiarse nunca de ningun compresor por muy conocido que sea y raramente dos ***ASProtect*** van a ser igual, primero porque cada uno lo comprime con unas determinadas opciones que no otro y asi va el tema.

Reventando el ASProtect..!!

Pues si chabales vamos a por el, imagino que los pasos estan claros, las teorias (de ***CrackslatinoS***) lo dejan bien claro, pero como estos tutos estan orientados para los que empiezan no va mal repasarlos para este programa concreto, tarnquilos que todos llegaremos al final con el programa desempaquetado ;XDD que malos somos, jeje..

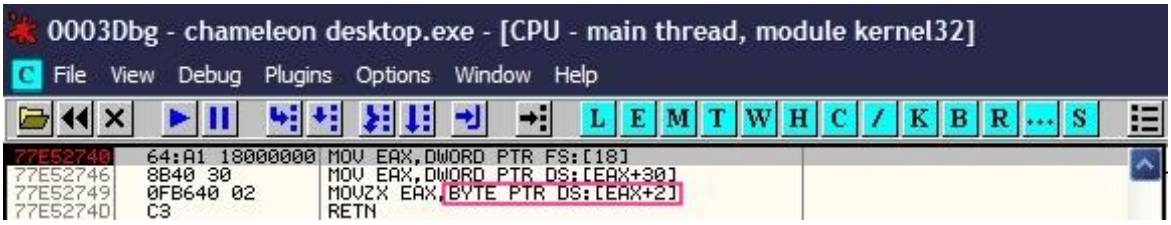
Como siempre carguemos el exe en ***OllyDbg*** que todos tenemos, si no no es dificil encontrarlo en la red, pasate por el ***FTP*** de ***Ricardo*** que creo que alli estara seguro, si no lo teneis bajaros tambien el plugin IsDebuggerPresent, el una dll que deberemos poner en la misma carpeta del Olly, esto es opcional porque se puede superar manualmente pero asi es mas rapido, si alguien no sabe superar el antidebug de ASProtect y no encuentra el plugin o lo que sea mirad como lo podeis hacer facil:

1.- Estamos en el ***EP*** (ojo no el OEP) vamos al commandBar ¿no tienes el plugin? bajalo tambien este te hara falta y tecleamos ***BP IsDebuggerPresent*** para para en la api cuando esta sea llamada por ***ASProtect***:

Si no dice nada al lado es que esta puesto el BreakPoint, si pone ***Unknown Identifier*** como en la imagen:

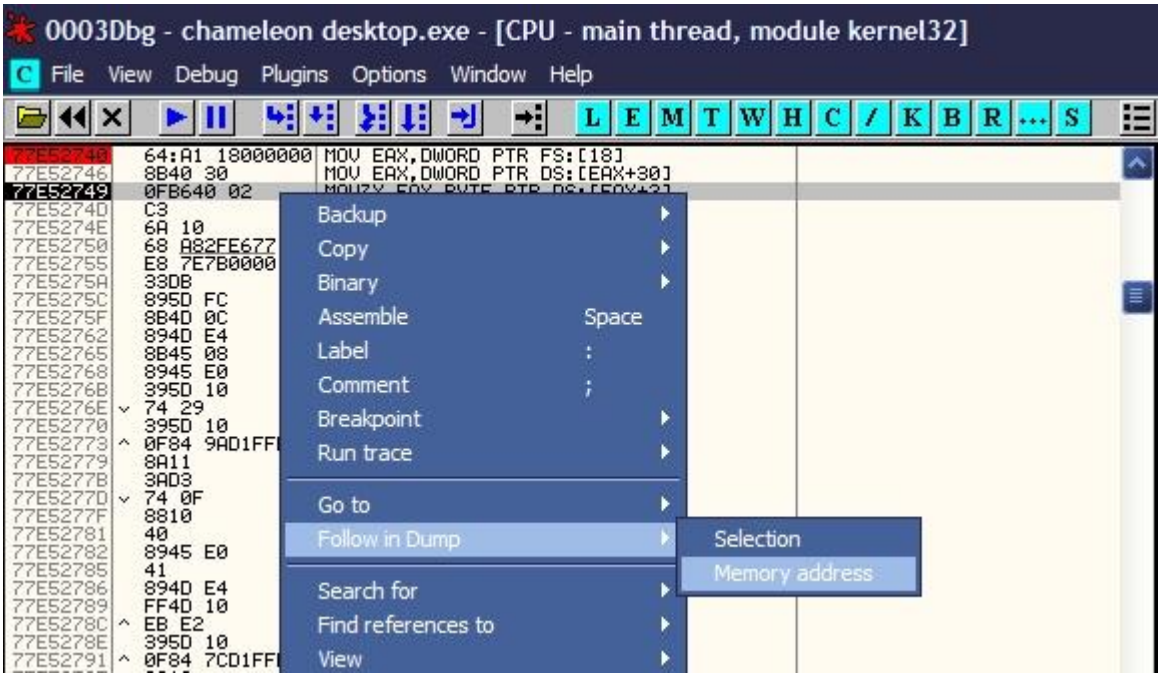
Significara que el nombre esta mal puesto, fijaos que puse ***Preset*** y no ***Present*** al final he ahi el error ¿ok?

2.- Damos RUN (***F9***) e iremos superando las excepciones hasta parar en la api:



Hemos parado en la api, esto lo podemos confirmar en la barra de estado de Olly, a ver que dice:

Hemos parado debido al ***BP IsDebugger***.. perfecto, en la imagen anterior seleccione el valor de ***[EAX+2]*** porque este es el que a nosotros nos interesa, el progama mra su valor si encuentra alli un ***1*** es que esta corriendo bajo un debugger y se cierra, pero si le cambiamos este valor a ***0*** creera que esta corriendo solo y se ejecuta, todo esto lo hace el plugin por nosotros si podeis bajaoslo ya que estar cambiando a cada momento el valor de ***[EAX+2]*** no es nada interesante y nos hara perder tiempo, la api en si son tres lineas quitando el ***RET*** final, lo que en realidad hace es leer esa posicion de memoria que ya tiene preparada de antemano, veamos como descubrir que direccion es, tracear hasta la linea ***MOVZX EAX, BYTE PTR [EAX+2]*** y desde alli hacer un Follow in Dump como en la siguiente imagen:



Ahora solo tenemos que echar un vistazo en la ventana ***Dump*** y descubriremos la direccion de memoria para nuestra maquina (perdonad que no ponga acentos pero es que el Word me va faltaaaaaaaal)

Si miramos alli en la citada ventana dump veremos la direccion y el byte de deteccion que recordad debe estar siempre a 0 para que no nos detecten el debugger:

```
Address   Hex dump                                          ASCII
7FFDF002  00 00 FF FF FF FF 00 00 40 00 A0 1E 24 00 00 00  ..     ..@.à▲$...
7FFDF012  02 00 00 00 00 00 00 00 14 00 80 4D FB 77 DE 55  ☻.......¶.ÇM¹wÞU
7FFDF022  F6 77 90 56 F6 77 01 00 00 00 50 2A D1 77 00 00  ÷wÉV÷w☺...P*Ðw..
7FFDF032  00 00 00 00 00 00 00 00 00 00 00 00 00 00 50 4C  ..............PL
7FFDF042  FB 77 FF 0F 00 00 00 00 00 00 00 00 6F 7F 00 00  ¹w *........oÂ..
7FFDF052  6F 7F 88 06 6F 7F 00 00 FB 7F 00 10 FC 7F 00 20  oÂê♠oÂ..¹Â.►ⁿÂ.
```

Para que se vea mejor he remarcado la direccion ***:7FFDF002*** en mi maquina y el byte de deteccion que esta a 0 (es el primero comenzando por la izquierda) como vemos no hace falta que lo cambie porque ya esta a 0, esto es gracias al plugin que ya oculta el ***Olly*** el solito ;XDDD

Dicho esto ahora todos podemos hacer correr ***ASProtect*** en nuestra maquina, sigamos con el crackeo del ***Chameleon***, como decia antes estamos parado en el ***EP*** (EntryPoint) del programa empaquetado:

```
0003Dbg - chameleon desktop.exe - [CPU - main thread, module chameleo]
File View Debug Plugins Options Window Help
L E M T W H C / K B R ... S

00401000  68 01905700   PUSH chameleo.00579001
00401005  E8 01000000   CALL chameleo.0040100B
0040100A  C3            RETN
0040100B  C3            RETN
0040100C  3F            AAS
0040100D  46            INC ESI
```

Aqui nada de ***PUSHAD*** ni pijadas de estas hay que currarselo, afortunadamente conocemos el método de ***Ricardo*** (hostias va el acento otra vez, jeje..) de ir superando excepciones hasta llegar a la ultima y ahi poner un ***BPM*** en toda la primera seccion para que ***Olly*** se detenga cuando el programa salte hacia alli, esto logicamente lo hara cuando se haya desempaquetado completamente y digamos ***ASProtect*** se retira y le pasa el control a nuestra aplicacion, en este momento el salto se produce justo a nuestro ***OEP***..!! ahí queremos estar nosotros, ojo que algunos compresores como este, ***PeLock***, ***ACProtect*** nos dificultan la labor añadiendo los conocidos ***Stolen Bytes*** ¿Que son los Stolen? pues son las primeras sentencias del programa original que en lugar de estar allí el típico:

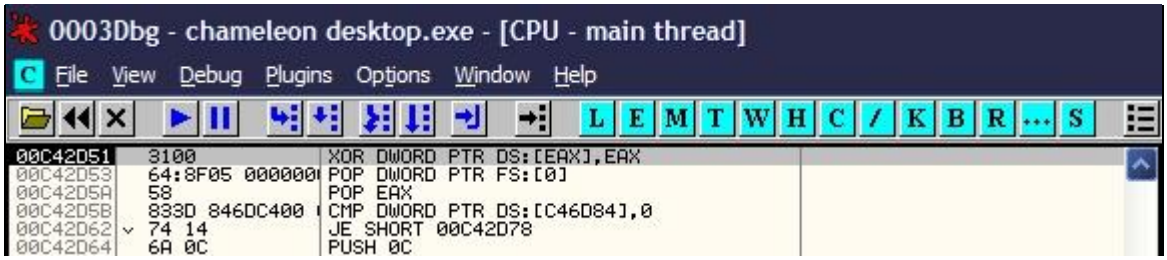
PUSH EBP
MOV EBP, ESP
....

Vemos que hay una ristra de ***00's*** en su lugar ¿? eso son los ***Stolen Bytes***, inconfundibles, he de reconocer que a mi esta idea me desconcertó en un inicio y no fueron pocas consultas las que tuve que hacer para aclarar un poco el tema, grácias ***Ricardo***..!! pero hoy por hoy hay muy buenos tutos escritos por la peña de la lista ***CrackslatinoS*** que son francamente buenos, en realidad grácias a todos..!! porque de ellos aprendo cada día un poco más, de eso se trata de compartir lo que sabes y escribir lo que vas aprendiendo de otros, con el tiempo ya iremos sacando cosillas propias por eso no hay que preocuparse llegará sólo. Creo que el tema de los Stolen está claro no? Busquemos la última excepción, debemos tener el cuadro de Excepciones (***Alt+O***) de esta manera configurado:

```
Debugging options
Commands | Disasm | CPU | Registers | Stack | Analysis 1 | Analysis 2 | Analysis 3
Security | Debug | Events | Exceptions | Trace | SFX | Strings | Addresses

[x] Ignore memory access violations in KERNEL32
Ignore (pass to program) following exceptions:
    [ ] INT3 breaks
    [ ] Single-step break
    [ ] Memory access violation
    [ ] Integer division by 0
    [ ] Invalid or privileged instruction
    [ ] All FPU exceptions
[ ] Ignore also following custom exceptions or ranges:
    0EEDFACE                                  Add last exception
    0EEDFADE
    406D1388                                  Add range
    80000002 (DATATYPE MISALIGNMENT)
    80000003 (BREAKPOINT)                     Delete selection
    80000004 (SINGLE STEP)
```

Esto es para garantizar que olly se detenga en todas las excepciones, si no lo tenemos así cargará directamente el programa sin detenerse ;(

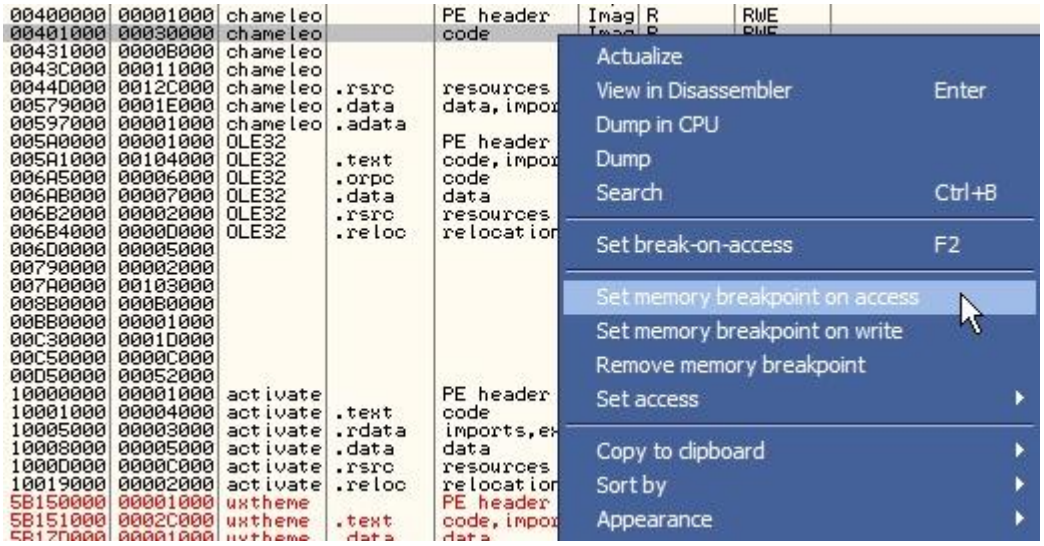
Vamos.. desde el ***EP*** picamos una sóla vez a ***F9***, tal y como se vayan sucediendo las excepciones las vamos superando con ***Sfift+F9***, así hasta que llegue un momento que al superar la excepción salga el programa en pantalla, esta es nuestra última excepción, la anotamos:



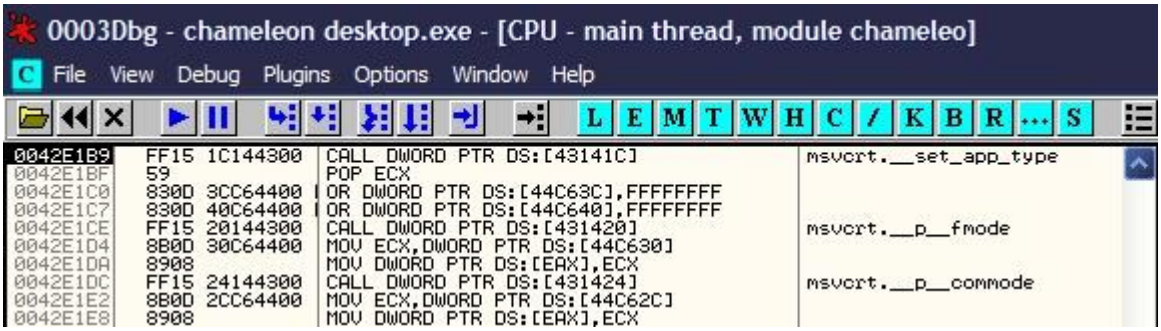
Ya tenmos el primer fato importante:

<u>Última excepción</u>: C42D51

Reiniciemos todo con ***Ctrl+F2*** y lleguemos hasta la última excepción otra vez, pero ahora no hagamos nada, pongamos un ***BPM*** en la primera sección, picamos ***Alt+M*** y luego hacemos:



La trampa está tendida, retornemos a la ventana de código (***Alt+C***) y ahora sí superemos la excepción con ***Shift+F9***, caeremos de pleno en el ***OEP***..!!



Algunos os preguntaréis ¿este es el ***OEP***? si es un ***CALL*** a la ***IAT***..!! Amigos necesitaremos un poco más de tiempo porque esto es un caso claro y evidente de ***STOLEN BYTES***, jeje.. antes lo decimos y mirad por donde aparecen, es lo que os decía al principio, las sentencias originales del programa se has sustituido por 00's, como se muestra en la siguiente imagen

```
0003Dbg - chameleon desktop.exe - [CPU - main thread, module chameleo]
File View Debug Plugins Options Window Help

0042E1A2  0000          ADD BYTE PTR DS:[EAX],AL
0042E1A4  0000          ADD BYTE PTR DS:[EAX],AL
0042E1A6  0000          ADD BYTE PTR DS:[EAX],AL
0042E1A8  0000          ADD BYTE PTR DS:[EAX],AL
0042E1AA  0000          ADD BYTE PTR DS:[EAX],AL
0042E1AC  0000          ADD BYTE PTR DS:[EAX],AL
0042E1AE  0000          ADD BYTE PTR DS:[EAX],AL
0042E1B0  0000          ADD BYTE PTR DS:[EAX],AL
0042E1B2  0000          ADD BYTE PTR DS:[EAX],AL
0042E1B4  0000          ADD BYTE PTR DS:[EAX],AL
0042E1B6  0000          ADD BYTE PTR DS:[EAX],AL
0042E1B8  00FF          ADD BH,BH
0042E1BA  15 1C144300   ADC EAX,chameleo.0043141C
0042E1BF  59            POP ECX
```

pero estas sentencias que faltan se deben ejecutar..!! ¿qué pasa entoneces con ellas? esto es muy sencillo, la rutina de descompresor hace algo así:

Descomprime código
. . .
Descomprime más código, jeje..
. . .
Ejecuta los Stolen Bytes <- ***Aqui queremos estar!!***
Salta al falso_OEP

Está claro no? antes de pasar el control a la aplicación debe ejecutar los bytes en alguna parte, esto sucede en plena rutina de descompresión, suele ser al final almenos en los casos que yo he visto, hay veces que lo ponen fácil y otras que resulta casi imposible encontrar los bytes y hey que deducirlos de alguna manera (la pila será nuestra gran aliada en esos casos, ver tutos de ***[Juan Jose]*** donde trabaja muy bien este tema)

En la imagen anterior se vern los ***00's*** a los que hacíamos referencia, el call no se ve porque se ofuscó, jeje.. pero se ven claramente sus bytes ***FF151C144300*** (comprobarlo), esos son los stolen para después, yo pongo aquí el método o pasos que sigo siempre en estos casos, ahora trato la iat primero, como decía antes nos situamos en el ***falso_OEP*** que es un call a la iat precisamente ya nos va bien y todo iremos más rápido, hagamos un Follow in ***Dump*** y miremos en la pria ventana dump valga la redundancia, ahora vamos mirando hacia arriba buscando el inicio de la tabla, es fácil darse cuenta porque es el inicio de la sección imagino que .rdata, .idata o alguna de estas porque ***ASProtect*** les borra el nombre, estoy da igual, lo que importa es donde empieza la tabla (iat):

```
Address   Hex dump                                          ASCII
00431000  D6 27 DA 77 B1 63 DB 77 9A 22 DA 77 10 24 DA 77  Í' rw▒c▄wÜ" rw►$ rw
00431010  D8 17 DA 77 00 00 00 00 60 12 00 10 00 00 00 00  Ï↨ rw....`↕.►....
00431020  AF 55 C4 77 7C 21 C4 77 3D 51 C4 77 9D 88 C4 77  »U─w|!─w=Q─w▯ë─w
00431030  F1 86 C4 77 0E 6C C4 77 70 2B C4 77 67 47 C4 77  ±å─w♫l─wp+─wgG─w
00431040  70 91 C4 77 E0 20 C4 77 D1 36 C4 77 6E 3C C4 77  pæ─w▯ ─w▯6─wn<─w
```

Este es nuestro inicio, como siempre lo anotamos:

<u>Inicio iat</u>: 431000

De la misma forma buscamos el final, esto puede resultar a veces algo confuso porque no se ve tan claritas las cosas ahora os diré como fdiferenciar si un valor está dentro de la iat o no, de momento veamos el final:

```
Address   Hex dump                                          ASCII
00431540  DC 79 D1 77 76 64 D3 77 E9 1C D3 77 00 00 00 00  ▬yÐwvdËwÚ└Ëw....
00431550  0A 6C 1B 76 4B 97 1B 76 E6 5D 1B 76 7D 01 1C 76  .[+vKù+vµ]+v}Ø└v
00431560  89 06 1C 76 00 00 00 00 77 21 5B 00 00 00 00 00  ë♠└v....w![.....
00431570  B0 FC 41 00 78 15 43 00 11 01 00 00 00 03 00 00  ▒ⁿA.x§C.◄Ø...♥..
00431580  5F 04 00 00 5F 04 00 00 0C 00 00 00 A0 10 40 00  _♦..._♦......á►@.
```

<u>Final iat</u>: 43156C

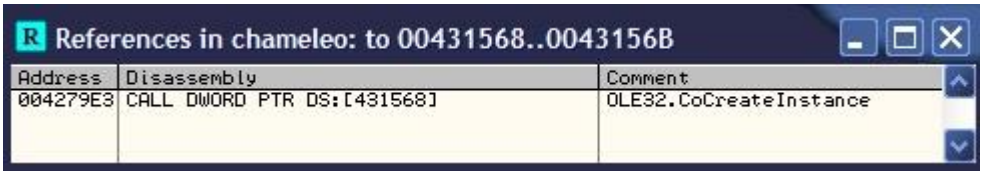
Con estos dos valores obtenemos fácilmente su longitud: 43156C-431000=56C

<u>Longitud_iat</u>: 56C

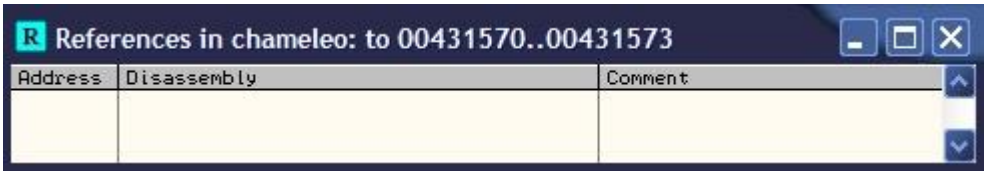
Y ahora lo que os decía, mirad esta última imagen como hemos determinado el final de tabla en :43156C pero hacia abajop siguen habiendo valores ¿como sabemos que no son de la iat? porque toda entrada de la iat está referenciada desde algún punto del programa con un salto o call del tipo:

jmp dword ptr [xxxxxxxx]
calll dword ptr [xxxxxxxx]

donde [xxxxxxxx] representa una dirección de la iat precisamente, entonces si seleccionamos una entrada dudosa y buscamos sus referencias con ***Ctrl+R*** sabremos rápidamente forma o no parte de la tabla, por ejemplo seleccionemos los bytes de la entrada :***431568*** (bytes: 77 21 5B 00) y pulsemos ***Ctrl+R***, veremos esto:



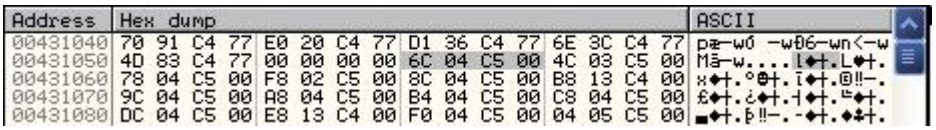
Esta entrada es referenciada desde ***:4279E3*** donde hay un ***jmp dword ptr [431568]***, sabemos que ***:431568*** está dentro de la iat y además es la entrada que estamos mirando, o sea que esta entrada si forma parte de la iat ¿ok? ahora miremos más abajo, por ejemplo en ***:431570*** (bytes: B0 FC 41 00) y hagamos los mismo:



¿lo véis? ninguna referencia, si hacemos esto con los valores que van hacia abajo no encontramos referencia alguna, o sea que el final de tabla se encuentra en ***:431568*** que es la última entrada referenciada, por mayor seguridad se le suele dar una entrada de más, imagino que es para que el ***Import Reconstructor*** (***ImpRec***) la reconozca bien pero esto no lo sé con seguridad.

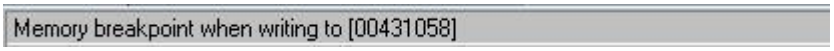
Ahora que hemos delimitado bien la tabla tenemos otra misión: buscar el ***call semi-mágico***, se le dice así porque rara vez repara la tabla entera, lo que si consigue es que muchas entradas no sean redirigidas que ya es bastante, si vamos al ***ImpRec*** con estos datos nos dirá que hay ***30 entradas*** sin resolver (realmente no son muchas, he visto casos en que hay más de ***100***) pero igualmente repararlas a mano no es un trabajo grato, necesitamos pues la ayuda del citado call, para localizarlo se puede hacer de esta manera:

1.- Le echamos la foto a algún puntero que esté redirigido, por ejemplo este:



nos centraremos en el que está seleccionado ***[431058]=00C5046C*** (recordar que los bytes se leen al revés), esta entrada nos hará digamos de guía por nuestra acelerada carrera, jeje.. reiniciamos, venga ya sabemos ***Ctrl+F2*** y estando en el ***EP*** en la ventana ***Dump*** pulsamos ***Ctrl+G => Go*** quiere decir y ponemos la dirección ***:431058***, aceptamos y caemos allí, le ponemos un ***BPM On Memory Write***, no pongo fot porque esto lo sabemos hacer ya, otra cosa cuando hayamos puesto el ***BPM*** para que no se nos haga muy trabajoso tener que estar superando excepciones lo que yo hago e imagino que todos también es ir al cuadro de excepciones que vimos antes con ***Alt+O*** y tildarlas todas para que no pare, tampoco adjunto imagen porque ya antes la vimos y lo que hay que hacer es realmente sencillo no tiene pérdida, o si, jeje..

Ahora damos RUN y Olly se detiene, miremos la barra de estado:



Yeeeeeep..!! que está escribiendo allí algo, miremos donde nos encontramos:

```
0003Dbg - chameleon desktop.exe - [CPU - main thread]
File View Debug Plugins Options Window Help
00C38F7A   881C3E     MOV BYTE PTR DS:[ESI+EDI],BL
00C38F7D   FF45 F0    INC DWORD PTR SS:[EBP-10]
00C38F80   FF4D EC    DEC DWORD PTR SS:[EBP-14]
00C38F83 ^ 75 95      JNZ SHORT 00C38F1A
00C38F85   5F         POP EDI
00C38F86   5E         POP ESI
00C38F87   5B         POP EBX
00C38F88   8BE5       MOV ESP,EBP
00C38F8A   5D         POP EBP
```

Aqui se detuvo Olly, si miramos la entrada los bytes no coinciden con los que tenemos anotados y seran los bytes finales, asi que demos RUN nuevamente para que continúe..hay que dar tres o cuatro veces mas prque esta escribiendo byte a byte y son lógicamente 4 bytes los que tiene la entrada, al siguiente RUN caemos aqui:

```
0003Dbg - chameleon desktop.exe - [CPU - main thread]
File View Debug Plugins Options Window Help
00C4266B   50         PUSH EAX
00C4266C   E8 53FCFFFF CALL 00C422C4
00C42671   E8 8AFEFFFF CALL 00C42500
00C42676   8B17       MOV EDX,DWORD PTR DS:[EDI]
00C42678   8902       MOV DWORD PTR DS:[EDX],EAX
00C4267A v EB 7E      JMP SHORT 00C426FA
00C4267C   83FB 01    CMP EBX,1
00C4267F v 74 05      JE SHORT 00C42686
00C42681   83FB 04    CMP EBX,4
00C42684 v 75 37      JNZ SHORT 00C426BD
00C42686   8A06       MOV AL,BYTE PTR DS:[ESI]
```

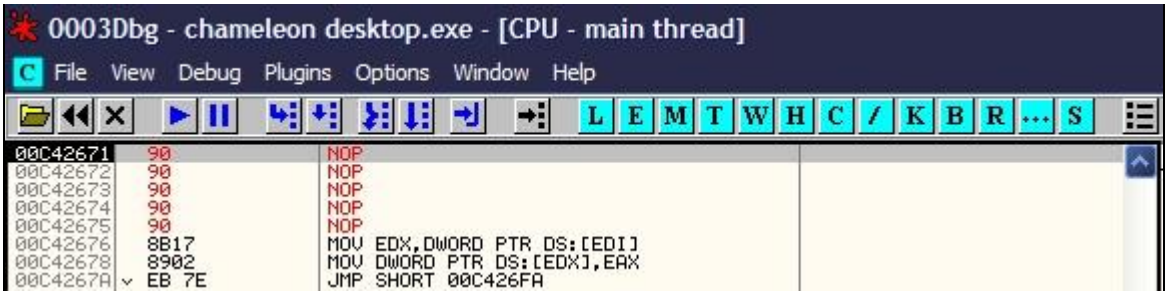
No os suena a otros ***ASProtect***? esos dos calls seguidos, el segundo es el ***MÁGICO***..!! es facil saberlo por dos cosas: por los tutos que ya hemos leido y porque si ponemos un ***Breakpoint HE*** en ***:C4266C*** encontramos un valor api correcto en ***EAX*** que pierde tras pasar por el siguiente call, luego este es nuestro magico, jeje.. no ha sido dificil, más bien fácil diría yo, vamos a ponerle solución a esto: reiniciamos, activamos las excepciones destildandolas, damos ***RUN***, al llegar a la primera excepción las volvemos a desactivar tildando las casillas, esto es porque desde el ***EP*** no tenemos acceso al call porque esta zona se crea en tiempo de ejecución y directamente no existe, no lo intentéis que ***Olly*** sirá que no puede no se qué, en fin lo hacemos desde la primera excepción y ya está, ***Ctrl+G*** y ponemos la dirección del call que es ***:C4266C*** pero ponemos como ***0C4266C*** con un ***0*** delante sino ***Olly*** se quejará y no hará nada, esto me tuvo loco una vez lo reconozca y gracias a un tuto porque sino aun lo estoy investigando ;-) una vez en el call le ponemos un ***BPX*** así:

```
0003Dbg - chameleon desktop.exe - [CPU - main thread]
File View Debug Plugins Options Window Help
00C4266B   50         PUSH EAX
00C4266C   E8 53FCFFFF CALL 00C422C4
00C42671   E8 8AFEFFFF CALL 00C42500
00C42676   8B17       MOV EDX,DWORD PTR DS:[EDI]
00C42678   8902       MOV DWORD PTR DS:[EDX],EAX
00C4267A v EB 7E      JMP SHORT 00C426FA
```

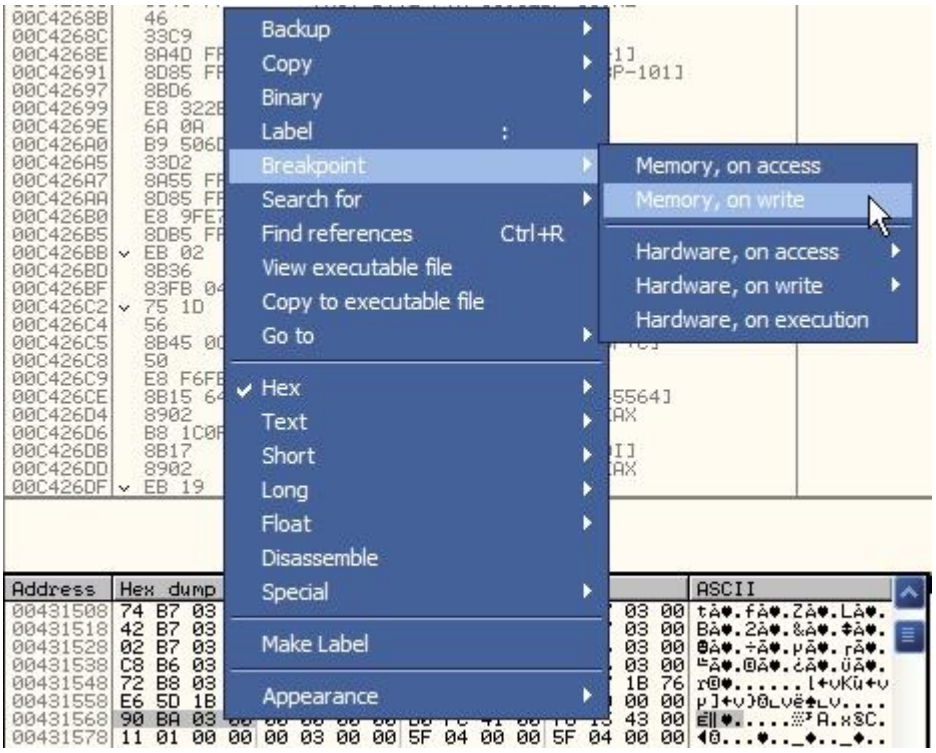
Y ahora inportante!! no demos ***F9*** sino ***Shift+F9***..!! un fallo de estos nos deja fuera! recordad que estamos detenidos en una excepción:

```
Access violation when writing to [00000000] - use Shift+F7/F8/F9 to pass exception to program
```

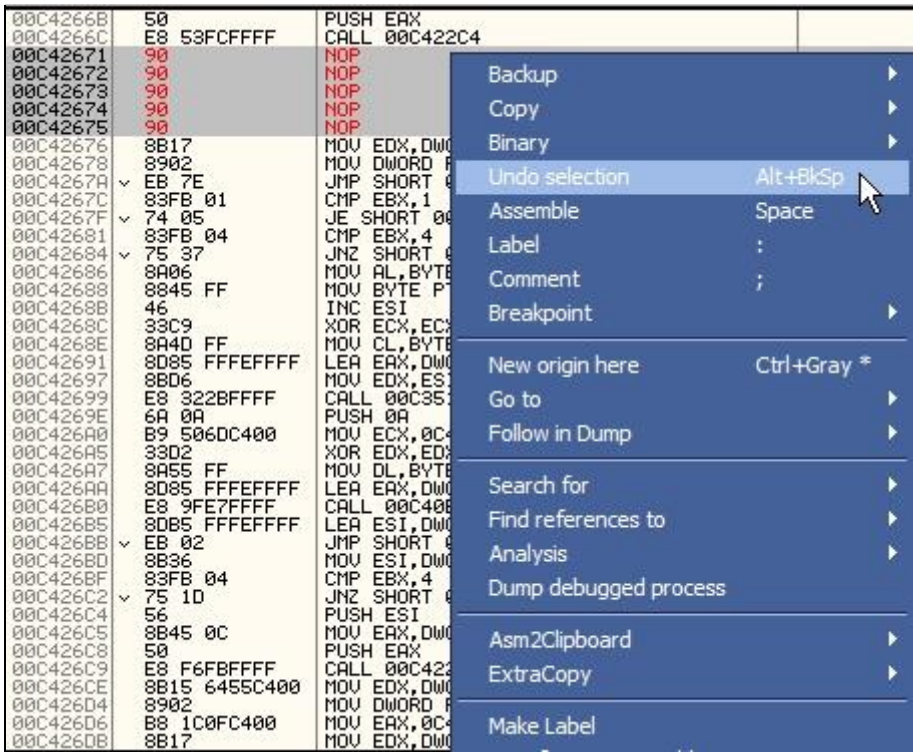
pero antes desactivemos las excepciones para llegar más rápido (ya sabéis como) y luego sí pulsamos ***Shift+F9***.. caemos sobre el ***call semi-mágico***, le damos su merecido ;-)



Así tiene que estar bien nopeado, jeje.. vamos al final de la tabla (el final real) pues sabemos que le dimos un pequeño margen de 4 bytes que ahora debemos obviar, el final de iat real está en ***:431568***, allí ponemos un ***BPM***



y damos ***RUN***, al momento parará, si miramos la barra de estado veremos que se detuvo por el ***BPM***, aparte ahora la tabla tiene otro aspecto mucho mejor, siguiente paso devolver el call nopeado a su estado original:



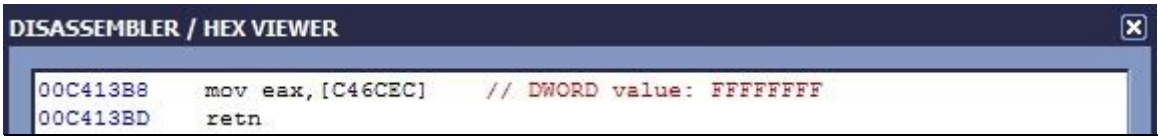
Seleccionando todos los bytes nopeados y haciendo un undo Selection como se ve en la imagen dejamos el call en su sitio, si no hacemos esto lo más probable es que no lleguemos al ***OEP***, sólo hubo un caso de ***ASProtect*** que vi que llegaba al ***OEP*** con el call nopeado ¿un error? no lo se pero llegaba, con lo quejosos que son y los chequeos ***CRC*** que tienen aquél tendría un mal día, bueno ahora configuramos las excepciones para que ***Olly*** pare en ellas ya que las necesitamos otra vez, como sabemos llegar al ***OEP*** porque lo hemos hecho antes pues lleguemos, ya saben como no? sino mirad un poco más arriba que está explicado no lo puedo estar explicando todo dos veces pues no acabaríamos nunca, se entiende no?

Estamos ya en el ***OEP*** todos? pues venga continuemos que aún queda trabajo, vamos al ***Import*** y le damos los datos de antes para que nos diga como está la tabla:

Luego picamos ***Get Imports*** y... 4 entradas? esto no es nada hombre, veamo cuales son, iremos por pasos, picaremos en ***Show Invalid*** y nos irá mostrando las malas, luego con el botón derecho picamos y elegimos ***Disassemble / HexView*** e iremos viendo su código:

<u>Entrada 1</u>:

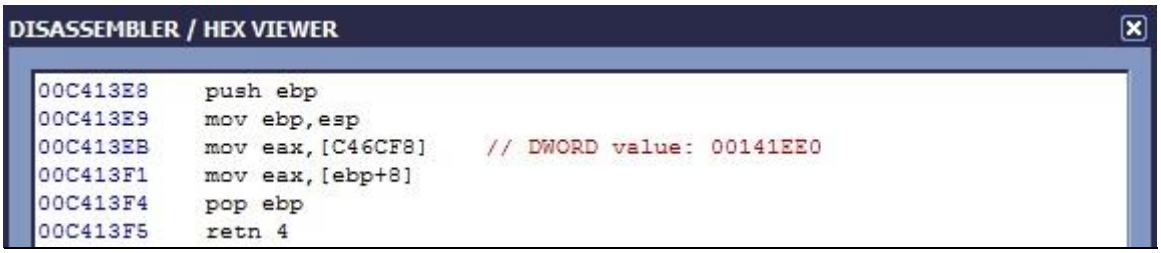
IAT rva: 3106C
Apunta a: C413B8



Cuando veáis esto directamente la ponéis como ***GetCurrentProcess***.

<u>Entrada 2</u>:

IAT rva: 31084
Apunta a: C413E8



Esta la reparamos como ***LockResource***.

<u>Entrada 3</u>:

IAT rva: 310AC
Apunta a: C40EF0

```
DISASSEMBLER / HEX VIEWER

00C40EF0    push ebp
00C40EF1    mov ebp,esp
00C40EF3    mov edx,[ebp+C]
00C40EF6    mov eax,[ebp+8]
00C40EF9    mov ecx,[C4543C]     // DWORD value: 00C45314
00C40EFF    mov ecx,[ecx]
00C40F01    cmp ecx,eax
00C40F03    jnz short 00C40F0E
00C40F05    mov eax,[edx*4+C45350]     // DWORD value: 00000000
00C40F0C    jmp short 00C40F15
00C40F0E    push edx
00C40F0F    push eax
00C40F10    call 00C35160     // = kernel32.dll/0191/GetProcAddress
00C40F15    pop ebp
00C40F16    retn 8
```

Esta sin lugar a dudas ***GetProcAddress***.

Entrada 4:

IAT rva: 310B8
Apunta a: C41360

```
DISASSEMBLER / HEX VIEWER

00C41360    push ebp
00C41361    mov ebp,esp
00C41363    mov eax,[ebp+8]
00C41366    test eax,eax
00C41368    jnz short 00C4137D
00C4136A    cmp dword ptr [C46978],400000
00C41374    jnz short 00C4137D
00C41376    mov eax,[C46978]     // DWORD value: 00400000
00C4137B    jmp short 00C41383
00C4137D    push eax
00C4137E    call 00C35158     // = kernel32.dll/016F/GetModuleHandleA
00C41383    pop ebp
00C41384    retn 4
```

Igual que la anterio, sin duda alguna ***GetModuleHandleA***.

Las podemos reparar todas con el plugin de ASProtect 1.22, a mi no me gusta utilizarlo y prefiero mirar yo porque hay veces que se equivoca ¿? si si se equivoca y no son pocas las veces, pero en este caso son pocas y bien conocidas ¿habéis visto ***GetVersion***? es una api casi siempre redirigida al igual que ***GetCommandLineA***, sigamos a lo nuestro, podemos reparar el dumpeado ya pero yo no lo hago así por tanto no lo repararemos, vamos a guardar la tabla tal cual reparada picamos en ***Save Tree*** y le damos un nombre, iat.txt por ejemplo, y ahora...

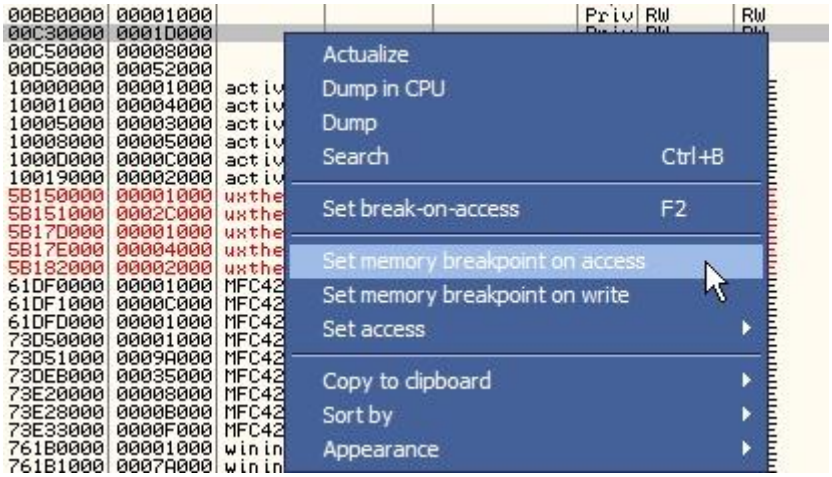
En busca del Arca Perdida... Los Stolen Bytes, jeje..

Vamos a abordar este tema con precaución y cordura, no se trata de venir y copiar os Stolen que yo de y quedarse tan agusto, NO SE TRATA DE ESO..!! se trata de que todos comprendamos cómo trabaja ASProtec este tema para poderlo combatir en otros programa que no sea este, porque creeis que hago el tuto tan explicado porque me gusta? pues NO sino porque pretendo enseñar lo poco que se y me gustaria que estos textos sean aprovechados para que vosotros mismos despues sin necesidad de tuto alguno llegueis a vencer las adversidades ¿de que vale copiar? DE NADA, venimos a aprender aqui ¿ok? queda alguien ahi? jeje.. lo digo por el bien de todos nosotros, lo importante de verdad es aprender, eso es lo que queda, vamos a por los put... Stolen..!!

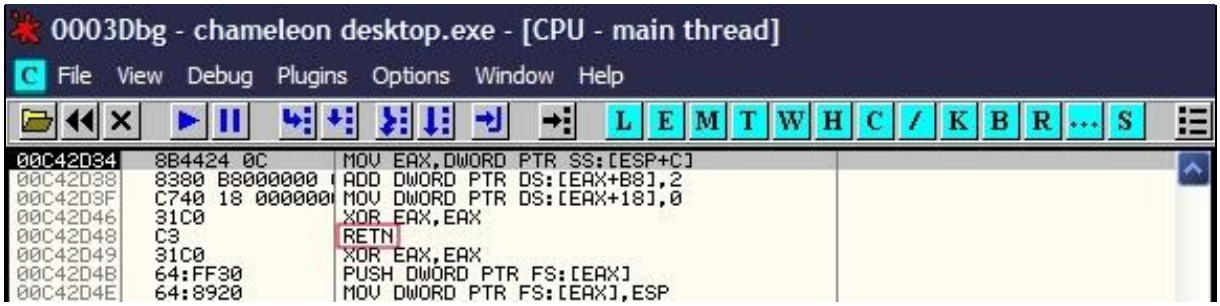
Reiniciamos por enésima vez, llegamos a la última excepción, miremos la pila (stack) a ver donde tiene ASProtect el manejador de excepciones (SE handler):

```
0012FF60  00C54D64  Pointer to next SEH record
0012FF64  00C42D34  SE handler
0012FF68  0012FF90
0012FF6C  00C30000
0012FF70  003E0000
0012FF74  00C57244  ASCII "¿¥ł"
```

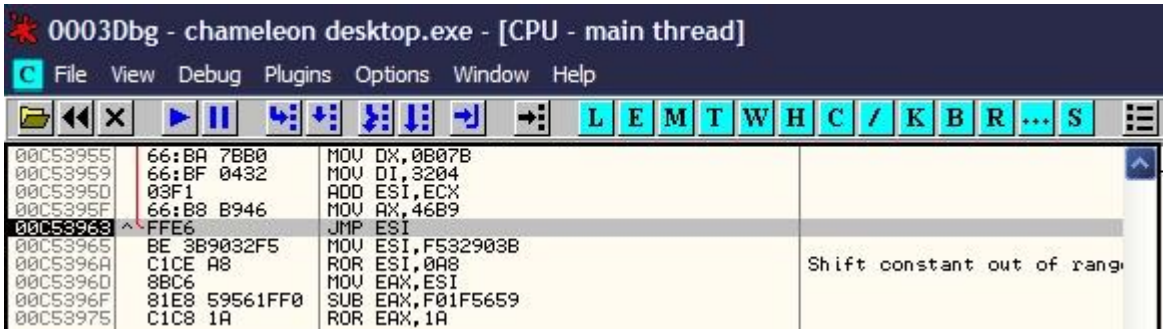
La tiene instalada en ***:C42D34***, esto quiere decir que al tratar la excepción retornaremos a esta zona, pongamos un ***BPM*** en la sección donde tiene la ***SEH***, damos a ***Alt+M*** y...



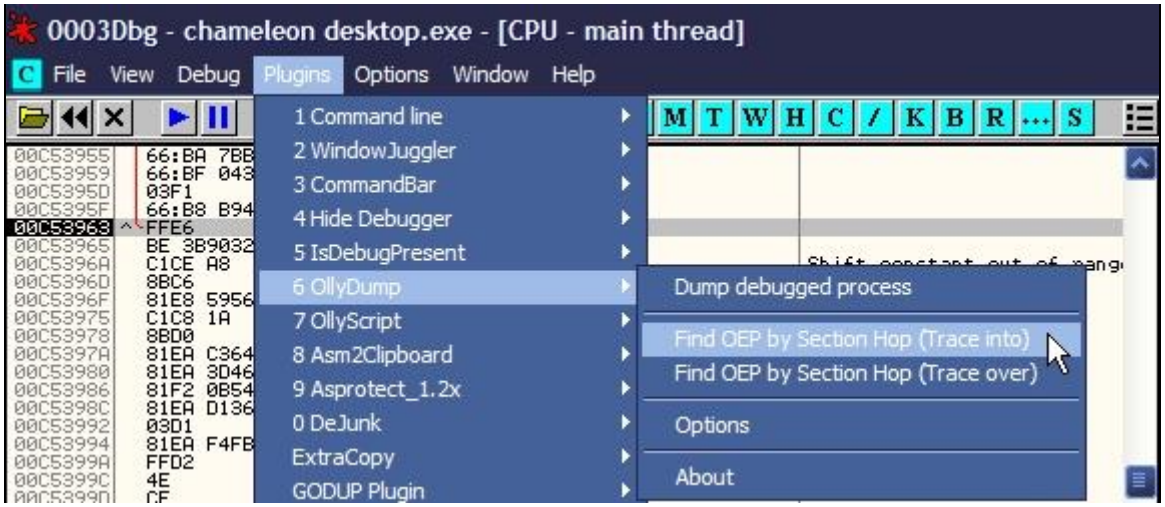
Allí lo ponemos on acces, mirad que la sección empieza en :C30000 y tien un tamaño de 1D000h bytes, luego termina en C30000+1D000=C4D000 y la SEH está en :C42D34 que cae dentro de esta sección, todo bien? sigamos, superamos la excepción con Shift+F9 como siempre y caemos como no en la zona de ASProtect ya que el BPM ha surgido efecto, veamos donde estamos:



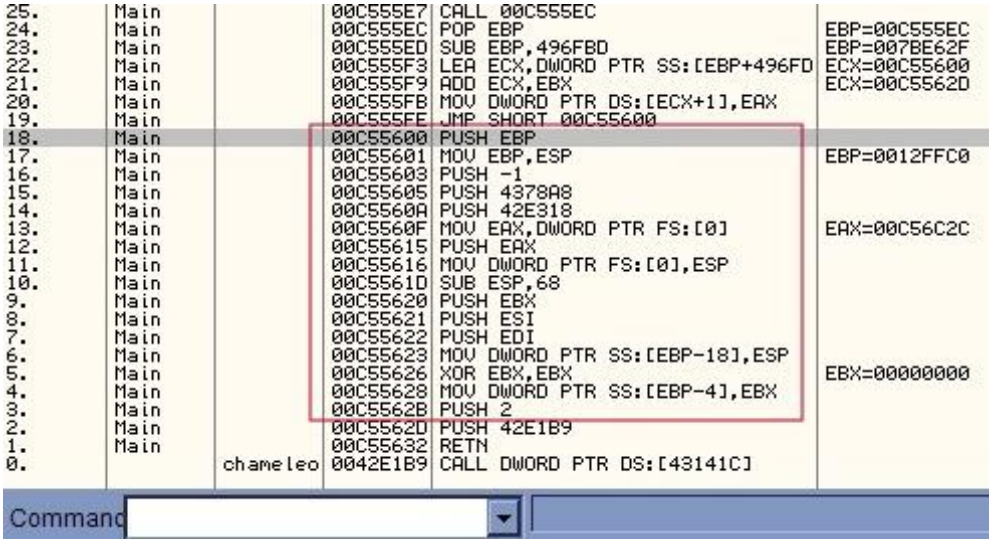
El ***RET*** seleccionado nos mete de pleno en ***ntdll***, nada damos a ***F9*** y como tenemos el ***BPM*** puesto retornamos a la seccion de ***ASProtect*** otra vez, yo sugiero llegar hasta el próximo ***RET*** con ***F8*** y a partir de allí ir traceando un poco con ***F7*** entrando en los calls, evitar algunos loops con un ***BPX*** al final + ***RUN*** y luego quitando el ***BPX*** claro así le comeremos terreno, bastante terreno, así lo hago siempre y creedme que funcionó en todos los ***ASProtect***!! lo que demuestra que el método es correcto, cuando no veamos loops a la vista llega el momento de crear una ***TRAZA***, normalmente se utiliza el valor que ***EAX*** tenía en el ***falso_OEP*** como condición de parada per en este caso no nos vale porque los últimos bytes cambian en cada ejecución, podríamos igual hallarlo porque lleva una bonita cadena asociada y siempre es la misma pero lo haremos de otra manera, antes de hacer nada picamos en el botón ... se abrira una ventana, con el boton derecho elegimos ***Log to file***, nos pedirá un nombre para el archivo, yo suelo poner ***traza_stolen.txt***, aqui salvará todo lo que hace ***ASProtect*** ¿no es bonito esto? monitorisazando al packer, jeje.. veréis que buenos resultados da, yo lancé la traza desde aquí:



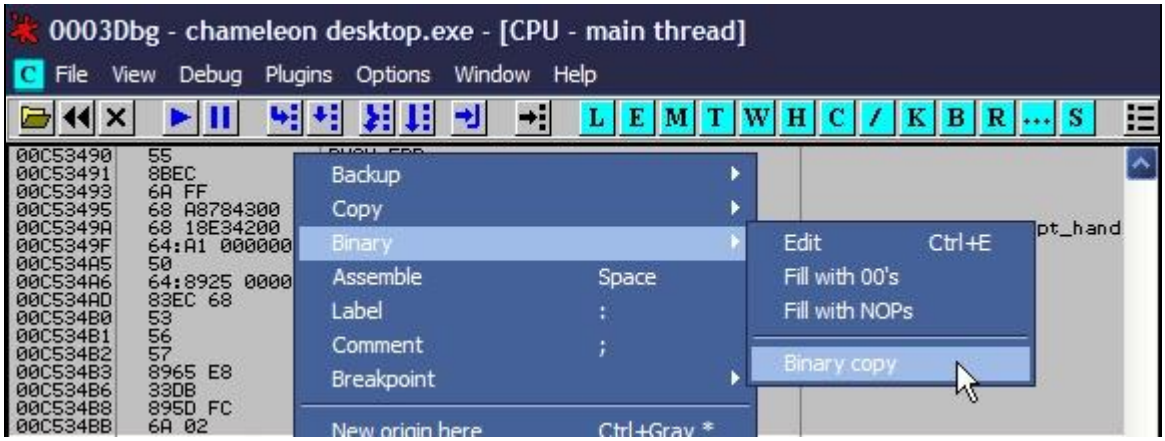
Lógicamente en tu máquina las direcciones cambiarán pero no el código..!! ahora vamos a iniciar la traza, como no tenemos condición de salida lo que haremos será un ***Trace Into***, esto se hace desde aquí:



Cuando pulsemos en ***Find OEP by Section Hop (Trace Into)*** el ***Olly*** quedará ***tracing...*** es aconsejable ir a la ventana log a la que se accede picando el botón que vimos de los puntos suspensivos y mirar que no estemos en un loop, nosotros como ya los habíamos evitado no tuvimos ese problema, si ocurriera abortamos la traza desde ***Debug->Open or Clear run trace***, salimos del loop y la lanzamos de nuevo, también es aconsejable en este caso de loops ir cerrando el archivo ***trace_stolen.txt***, eliminarlo y crear uno nuevo para que no recoja todos los bucles. En este caso esto no hace falta y ***Olly*** se detiene pronto, miremos en la ventana log ¿se dice asi? y vemos



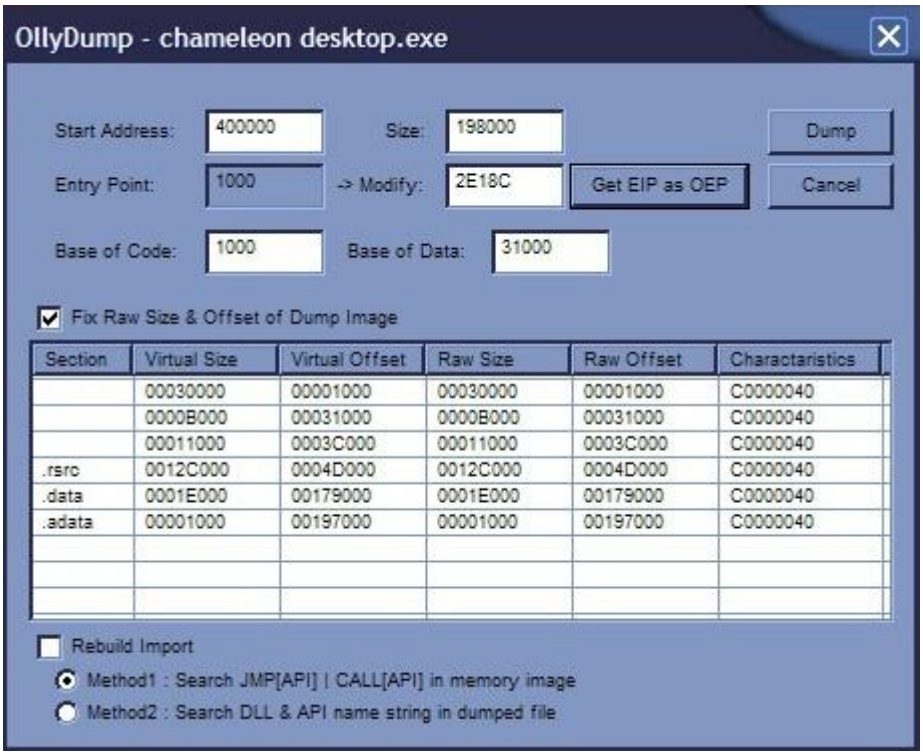
La line encierra todos los ***Stolen Bytes***, van desde el ***PUSH EBP*** hasta el ***PUSH 2*** incluido ambos, ahora sí que tenemos todo lo necesario, como empiezan en ***:C53490*** y los copiamos como se ve en la foto:



Una vez copiados vamos a la zona del ***falso_OEP*** y los pegamos, quederá así:

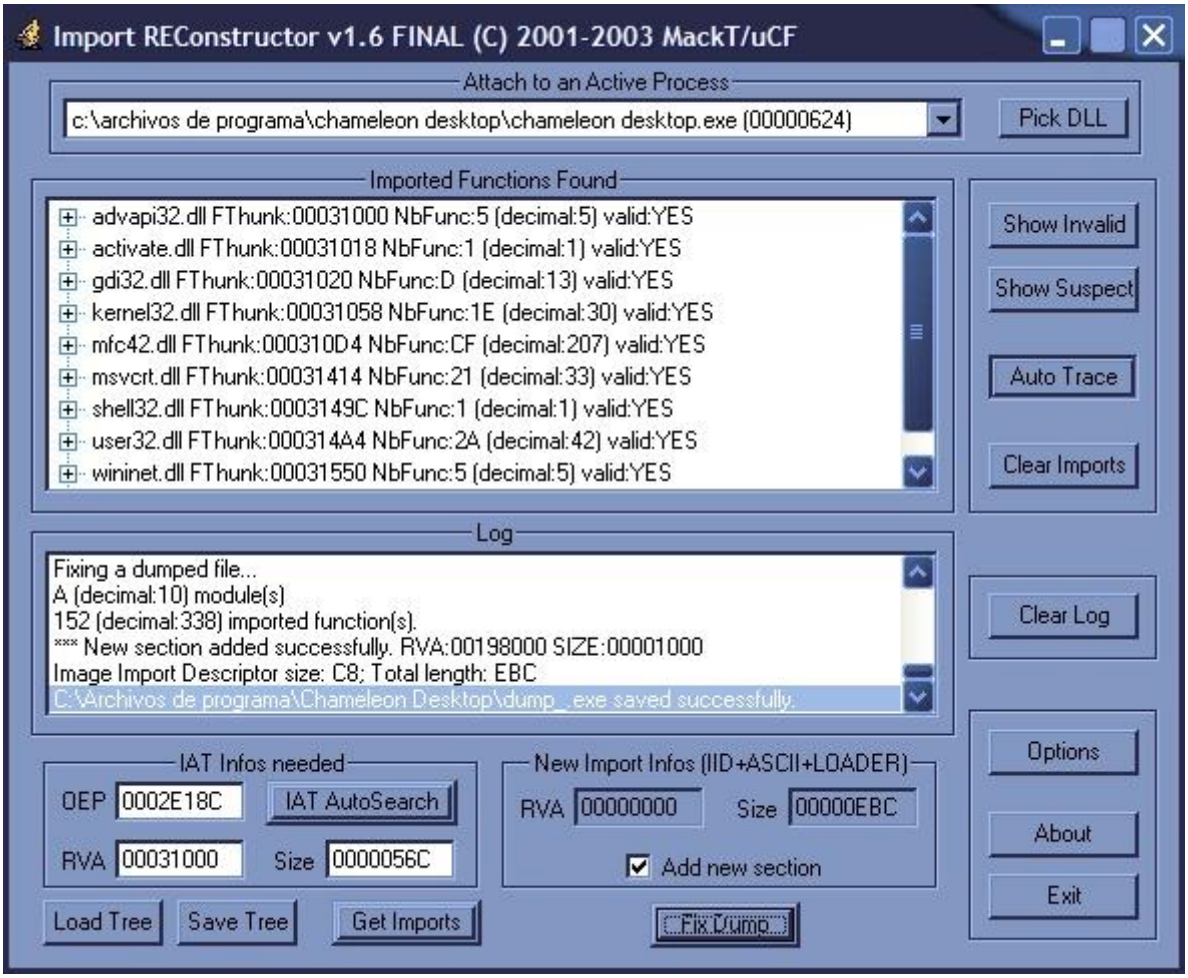
```
0003Dbg - chameleon desktop.exe - [CPU - main thread, module chameleo]
File View Debug Plugins Options Window Help

0042E18C  55              PUSH EBP
0042E18D  8BEC            MOV EBP,ESP
0042E18F  6A FF           PUSH -1
0042E191  68 A8784300     PUSH chameleo.004378A8
0042E196  68 18E34200     PUSH chameleo.0042E318                JMP to msvcrt._except_hand
0042E19B  64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0042E1A1  50              PUSH EAX
0042E1A2  64:8925 000000  MOV DWORD PTR FS:[0],ESP
0042E1A9  83EC 68         SUB ESP,68
0042E1AC  53              PUSH EBX
0042E1AD  56              PUSH ESI
0042E1AE  57              PUSH EDI
0042E1AF  8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
0042E1B2  33DB            XOR EBX,EBX
0042E1B4  895D FC         MOV DWORD PTR SS:[EBP-4],EBX
0042E1B7  6A 02           PUSH 2
0042E1B9  FF15 1C144300   CALL DWORD PTR DS:[43141C]            msvcrt.__set_app_type
0042E1BF  59              POP ECX
```

Ya está todo a punto para dumpear, jeje.. con lo ***Stolen*** en su sitio y todo, vamos a volcar con el propio ***OllyDbg***, es que tiene de todo..!! venga vamos para allá



Importante tener desmarcada la casilla ***Rebuild Import*** ya que no trabaja bien, después picamos en ***Get EIP as OEP*** y ***Dump*** para acabar, le damos un nombre dump.exe por ejemplo y terminamos con esto, ahora le toca el turno al ***ImpRec*** nuevamente ¿recordáis el archivo salvado con ***Save Tree***? cargamos el proceso con ***Attach to an Active Process*** y luego a ***Load Tree*** y elegimos ***iat.txt*** que teníamos guardado de antes, ojo que hay que cambiarle el valor del ***OEP*** ya que tiene puesta la dirección del falso ***:42E1B9*** que edberemos cambiar por ***:42E18C*** y luego ya sí podemo reparar como siempre, pinchamos en ***Fix Dump*** y seleccionamos el dumpeado que hemos llamado ***dump.exe***, cuando el Import termine nos habrá creado una nueva ***IT*** (Import Table), no hay que confundirla con la ***IAT*** (Import Address Table) que es otra cosa, pues si no estoy mal informado la ***IAT*** se rellena en función de la ***IAT***, digamos que la ***IT*** guarda la información necesaria para que se cree la ***IAT*** en condiciones, creo que guarda el nombre de las librerías (***dll's***) que importa el programa y el nombre de las funciones (apis) que también importa, todo esto no lo puedo asegurar ya que no lo he leído en ningún sitio pero por pura deducción creo que será así, bueno no me gustaría engañar a nadie por supuesto, asi que sigamos a lo nuestro, estábamos con el ***Import*** y la iat reparada, cuando termine tendremos en la misma carpeta de instalación del programa el archivo ***dump_.exe*** que es con el que vamos a trabajar a aprtir de ahora.

Como apunte final antes de pasar a otros aspectos mostraré una imagen de como debe quedar el ***Import*** después de reparar el archivo, notad la dirección válida del ***OEP*** y no la falsa y todas las entradas resueltas:



Y esto es todo en cuanto al ***Import Reconstructor*** que hizo un magnífico trabajo ;-)

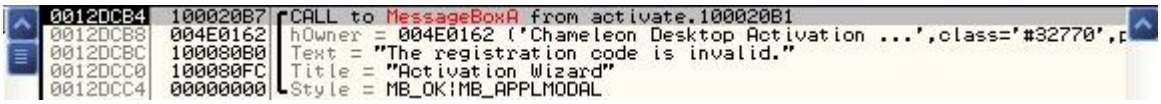
Si probamos el archivo ahora... (dump_.exe hay que ejecutar) ocurrirá lo siguiente:

Funciona...!!! jeje.. ya tenemos un exe funcional desempaquetado, ahora SI podemos atacarle desde Olly ya que los bytes que encontremos en memoria existirán en el disco ya que no está comprimido.

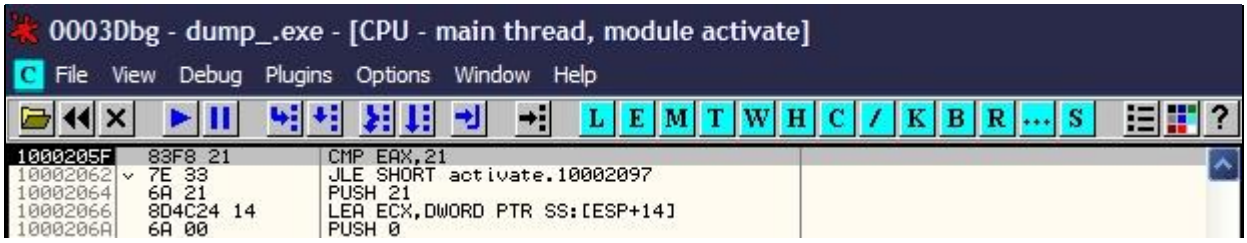
Registrando el producto...

Antes de atacar como locos intentemos Registrarnos ;-) pulsamos en Try y se abrirá un cuadro de registro donde nos pide un RegCode que está claro que no sabemos, pero bueno no nos vamos a quedar sin probar verdad?, asi que ponemos cualquier código que se nos ocurra y aceptamos, BOUUUUUUM..!!

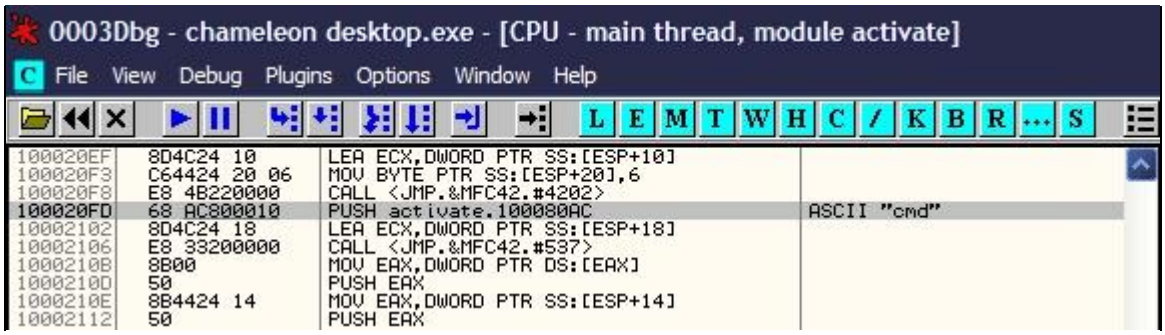
Pongamos un BP MessageBoxA en el CommandBar de Olly, volvemos a repetir la operación y ahora salta, es momento de mirar en el stack que suele haber información muy interesante y como esta:



Vaya vaya, así que la llamada se produjo en ***:100020B1***, pero esta dirección no forma parte de nuestro exe ¿? no pasa nada, si miramos al lado del ***exe*** vemos una bonita librería llamada ***activate.dll*** y es la responsable del mensaje, bien echemos un vistazo entonces antes del call que llama al mensaje, tenemos dos cosas, primero:



Si ponemos un ***BPX*** en esta parte vemos que ***EAX*** lleva el número de díigitos de nuestro ***RegCode*** y se compara con ***21h*** que serían ***33*** decimal, o sea que mira si tiene ***33*** dígitos, sino es así salta a otra parte donde mira si ***[EAX-8]*** vale ***21***, pongamos uno de ***33*** dígitos y aceptamos, rompemos en el ***BPX*** vemos que superamos la zona del mensaje de error, pero..!! volvemos a ver el ***Invalid***..!! más abajo sigue verificando cosas, la que más me llamó la ateción fue esta parte:



Parece buscar un ***cmd*** en el serial, pongámoslo, por ejemplo probemos con este:

cmd212121212121212121212121212123

aceptemos y... Vaya no ha dicho nada, se ha abierto otra pantallita en la misma ventana donde solicita un ***Activation Code***, jeje.. pongamos cualquier cosa y aceptemos:

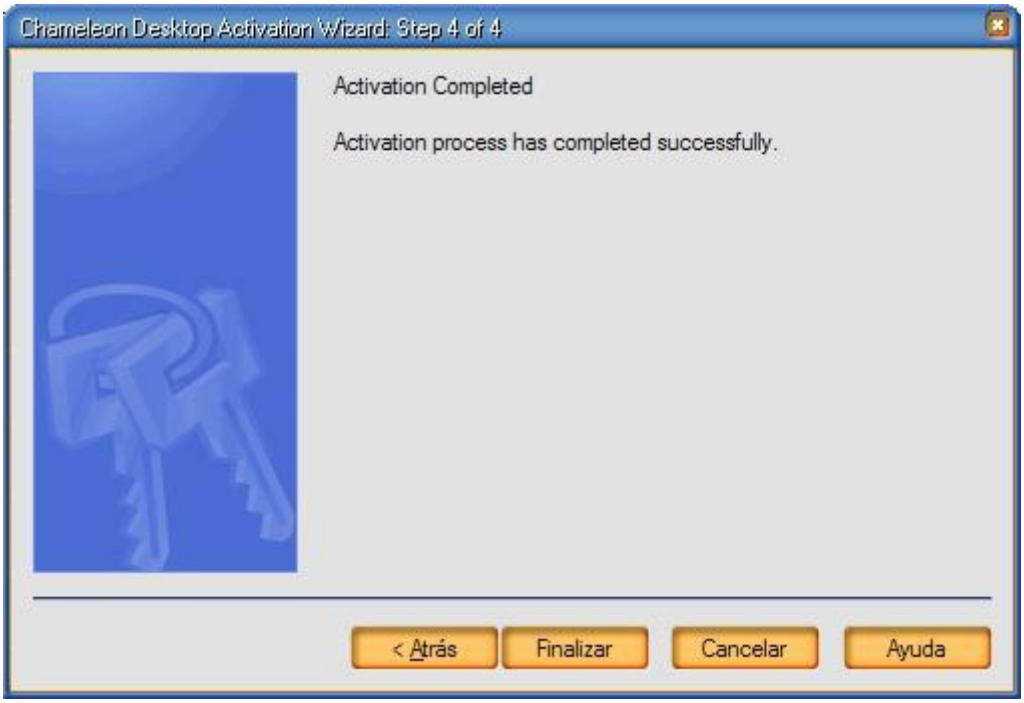
Era de esperar no? hagamos la misma opereción de antes del ***BPX*** y miremos al stack:

```
0012DC70  10002B75  CALL to MessageBoxA from activate.10002B6F
0012DC74  00840190  hOwner = 00840190 ('Chameleon Desktop Activation ...',class='#32770',p
0012DC78  100081E0  Text = "Activation code is invalid."
0012DC7C  100080FC  Title = "Activation Wizard"
0012DC80  00000000  Style = MB_OK|MB_APPLMODAL
```

Venga lo de antes pero pararemos en ***:10002B6F*** y miraremos a ver que hay por allí:

```
0003Dbg - dump_.exe - [CPU - main thread, module activate]
File View Debug Plugins Options Window Help

10002B5A   8378 F8 64      CMP DWORD PTR DS:[EAX-8],64
10002B5E   7D 2B           JGE SHORT activate.10002B8B
10002B60   8B4E 20         MOV ECX,DWORD PTR DS:[ESI+20]
10002B63   55              PUSH EBP
10002B64   68 FC800010     PUSH activate.100080FC          ASCII "Activation Wizard"
10002B69   68 E0810010     PUSH activate.100081E0          ASCII "Activation code is invalid."
10002B6E   51              PUSH ECX
10002B6F   FF15 B4520010   CALL DWORD PTR DS:[<&USER32.MessageBoxA  USER32.MessageBoxA
```

Igual que antes, ahora [***EAX-8]*** guarda también el número de dígitos de nuestro ***ActCode*** si es superior o igual a ***64h*** (100 decimal) saltamos or encima del mensaje, pues nada pongamos uno de ***100*** dígitos y probemos...



Aceptamos y sale este otro mensaje...

Será posible? no me creo nada, si ni siquiera lo ha validado como que ***"Thank You..."*** aceptamos y el programa se reinicia sólo, pues vale.. lo ejecutamos de nuevo y la nag en pantalla, ya decía yo que tan fácil no odía ser, está claro que ser cerró para validar el código en el próximo inicio del programa, podríamos rastrear esta opción pero lo haremos de otra manera mejor: Atacaremos las limitaciones directamente, sabemos que muesta una nagscreen,

si adelantamos el reloj ***EXPIRA***, o se que tiene ***Time-Trial***, y para colmo inserta un pequeño ***LOG*** en la parte superio de la imagen al salvarla, vamos por pasos ;-)

Asesinando la nag...

jeje.. bonito título, vamos a por ella por el método tradicional de traceo con ***F8*** hasta que aparezca en pantalla e investigar un poco, si hacemos esto llegamos al siguiente call que muestra la nag en pantalla:

```
0003Dbg - dump_.exe - [CPU - main thread, module dump_]
File View Debug Plugins Options Window Help
00401BE2  .  FF15 9CC1430(  CALL DWORD PTR DS:[43C19C]        dump_.00401940
00401BE8  .  6A 00          PUSH 0
00401BEA  .  8D4C24 18      LEA ECX,DWORD PTR SS:[ESP+18]
00401BEE  .  E8 6D010000    CALL dump_.00401D60
00401BF3  .  8D5424 14      LEA EDX,DWORD PTR SS:[ESP+14]
00401BF7  .  8D4C24 14      LEA ECX,DWORD PTR SS:[ESP+14]
00401BFB  .  C68424 94000(  MOV BYTE PTR SS:[ESP+94],3
00401C03  .  8956 20        MOV DWORD PTR DS:[ESI+20],EDX
00401C06  .  E8 37C00200    CALL <JMP.&mfc42.#2514>
```

Como la nag se pinta dentro le ponemos un ***BPX*** y reiniciamos, cuando paremos aqui entramos con ***F7*** y a ver que encontramos, ya no sabe uno lo que pensar:

```
0003Dbg - dump_.exe - [CPU - main thread, module dump_]
File View Debug Plugins Options Window Help
00401940  $  E8 1BFEFFFF    CALL dump_.00401760
00401945  .  85C0           TEST EAX,EAX
00401947  .v 75 0A          JNZ SHORT dump_.00401953        Bonito saltoooo...
00401949  .  50             PUSH EAX
0040194A  .  50             PUSH EAX
0040194B  .  E8 A06C0200    CALL dump_.004285F0             Call_Nagscreen..!!
00401950  .  83C4 08        ADD ESP,8
00401953  >  C3             RETN
```

Como marcan los comentarios el call_nag es el segundo, pero entremedio de ambos calls hay un bonito salto que de producirse entramos sin la nag..!! o sea que el ***call 401760*** debe retornar ***EAX=!0*** (***EAX=1*** por ejemplo ya nos va bien), lo mismo ahora ***BPX*** en ***:401940*** y cuando pare entremos..

```
0003Dbg - dump_.exe - [CPU - main thread, module dump_]
File View Debug Plugins Options Window Help
00401760  $v E9 16000000    JMP dump_.0040177B
00401765  .^ 72 C6          JB SHORT dump_.0040172D
00401767  .  A9 3F650DC9    TEST EAX,C90D653F
0040176C  .  BD 9CFD12AC    MOV EBP,AC12FD9C
00401771  .  691F B973694(  IMUL EBX,DWORD PTR DS:[EDI],4F6973B9
00401777  .  CC             INT3
00401778  .  91             XCHG EAX,ECX
00401779  .  0921           OR DWORD PTR DS:[ECX],ESP
0040177B  >  33C0           XOR EAX,EAX
0040177D  .  C3             RETN
```

Queeeeeeeeé, esto es una broma? el call es un salto directo a un ***XOR EAX, EAX*** que como sabemos hace ***EAX=0*** justo lo contrario de lo que necesitamos, es alucinante, entonces como vamos a entrar sin la nag? está claro que esto va a retornar siempre ***EAX=0*** si o si, de momento reiniciemos, vamos al ***call_tonto*** o ***call_inútil*** como queramos da igual, y cambiamos:

```
0003Dbg - dump_.exe - [CPU - main thread, module dump_]
File View Debug Plugins Options Window Help
00401760  $v E9 16000000    JMP dump_.0040177B
00401765  .^ 72 C6          JB SHORT dump_.0040172D
00401767  .  A9 3F650DC9    TEST EAX,C90D653F
0040176C  .  BD 9CFD12AC    MOV EBP,AC12FD9C
00401771  .  691F B973694(  IMUL EBX,DWORD PTR DS:[EDI],4F6973B9
00401777  .  CC             INT3
00401778  .  91             XCHG EAX,ECX
00401779  .  0921           OR DWORD PTR DS:[ECX],ESP
0040177B  >  33C0           XOR EAX,EAX
0040177D     40             INC EAX
0040177E     C3             RETN
```

de esta manera retornamos siempre con ***EAX=1***, si miramos las referencias vemos que el call es llamado hasta ***4*** veces, o sea que este cambio afecta a estas zonas de código, bueno salvemos los cambios, probemos el nuevo ***exe***.. eeeeeeeeei.. entramos sin la nag, vamos al ***About*** y...

JeJeJe, allí dice que estamos registrados, vaya parche más tonto de verdad, salvamos una foto y vemos que no pone el ***LOGO***, esto es increíble pero cierto, pensé que la lucha sería más fuerte que no un simple ***INC EAX*** tire por tierra todo el sistema defensivo de ***Chameleon*** que como dije al principio es un programa que me gustó bastante cuando lo vi.

Sólo me queda saludar a toda la gente de ***CrackslatinoS*** de los que voy aprendiendo cada día, la lista es enorme y crece de una manera desorbitada, jeje..

Y con esto damos ya por terminado el mini_tuto de hoy ya que los objetivos están cubiertos, para cualquier duda, pregunta, sugerencia, crítica o lo que queráis sentíos libres de escribirme a:

syxe00@yahoo.es
syxe00@hotmail.com

Por supuesto todos los mensajes serán contestados. Hasta la próxima